



23339 tegn

28 Sider

RAPPORT ROBOTBIL

Uddannelse
IT-Teknolog

KSB / 2026

Indhold

2. Sekundær forside	2
3. Resumé.....	3
5. Indledning.....	4
6. Metoder og værktøjer.....	5
7. Projektmiljøet.....	6
8. Krav	7
9. Systemdesign og Moduler	9
9.1.1 TOF-design.....	9
9.2.1 IR-design.....	10
9.3 UDP og Fodbold.....	13
9.4.1 Motor	14
9.5.1 Sumo design.....	15
9.6.1 Wall follow	17
9.7.1 Main design	18
10. Systemtest/integrationstest.....	22
11. Bevis	23
12. Projektledelse	24
13. Perspektiv / refleksioner.....	25
14. Konklusion	26
15. Appendiks.....	27



2. Sekundær forside

Rapporten er udarbejdet af Marcus, Mark, Christian og Malte. Alle fire medlemmer har bidraget til hele projektet og rapporten. Mark og Malte har primært arbejdet med programmering, mens Marcus og Christian har haft størst fokus på embedded-delen og rapportskrivningen.

Nedenstående forslag fordeler det individuelle ansvar på underafsnit i hvert fagligt hovedafsnit. Fordelingen skal afstemmes med de faktiske bidrag.

Forfatter	Forslag til ansvar for underafsnit
Mark	5.1 Funktioner og konkurrencer; 6.1 Python og PyCharm; 7.1 Gruppekompakt; 8.1 Software og fjernstyring; 9.3.1–9.3.3 UDP og fodbold; 9.7.1–9.7.3 Main; 10.1 Softwareintegration; 11.1 Dokumentation af fjernstyring; 12.1 Planlægning af softwareopgaver; 13.1 Refleksion over programmering; 14.1 Konklusion om software og fjernstyring.
Malte	5.2 Projektets formål; 6.2 Flowcharts og modulstruktur; 7.2 Arbejdsfordeling; 8.2 Krav til Wall Follow og SUMO; 9.5.1–9.5.3 SUMO; 9.6.1–9.6.3 Wall Follow; 10.2 Test af autonome funktioner; 11.2 Dokumentation af autonom kørsel; 12.2 Tidsforbrug på programmering; 13.2 Refleksion over autonome funktioner; 14.2 Konklusion om autonom kørsel.
Marcus	5.3 Komponenter og rammer; 6.3 WBS, Gantt-diagram og kritisk vej; 7.3 Forventningsafstemning; 8.3 Embedded-krav; 9.1.1–9.1.3 TOF-sensor; 9.4.1 Design af motorstyring; 10.3 Integration af sensorer; 11.3 Dokumentation af TOF-målinger; 12.3 Opdeling af opgaver i WBS; 13.3 Refleksion over konstruktion; 14.3 Konklusion om embedded-delen.
Christian	5.4 Problemformulering; 6.4 ClickUp og opfølgning; 7.4 Koordinering i gruppen; 8.4 Krav til projektledelse; 9.2.1–9.2.3 IR-sensor; 9.4.2–9.4.3 Implementering og test af motorstyring; 10.4 Samspil mellem motorer og styring; 11.4 Dokumentation af IR-sensor og motorer; 12.4 Tidsplan og Gantt-diagram; 13.4 Refleksion over samarbejde; 14.4 Konklusion om proces og læring.

Nummerering: De foreslåede underafsnit i afsnit 5–8 og 10–14 skal tilføjes i rapporten. Under Main skal implementering og test nummereres 9.7.2 og 9.7.3. Resumé og bilag er udarbejdet i fællesskab.



3. Resumé

Projektet handler om at udvikle en robotbil til konkurrencerne Wall Follow, SUMO og fjernstyret fodbold. Formålet er at undersøge, hvordan software, elektronik og mekanik kan arbejde sammen i en bil, der både skal kunne køre selv og fjernstyres.

Bilen er udstyret med motorstyring, afstands- og refleksionssensorer og et LiPo-batteri. Sensorerne skal bruges til den autonome kørsel, mens fjernstyringen skal foregå via Wi-Fi gennem et Python-program. I programmet skal brugeren også kunne skifte mellem bilens kørselsfunktioner og se batteriets aktuelle spænding.

Rapporten følger arbejdet fra planlægning og design til konstruktion, programmering og test. Vi beskriver vores valg undervejs og forklarer, hvorfor vi har valgt de enkelte løsninger. Softwaren er opdelt i moduler, så koden er lettere at overskue, teste og videreudvikle. Vi har også arbejdet med at effektivisere koden og håndtere de elektriske og mekaniske udfordringer, der er opstået undervejs. Testene skal vise, om bilens funktioner virker som planlagt, og hvor der er behov for forbedringer.



5. Indledning

I dette projekt er der blevet planlagt og udviklet en robotbil. Robotbilen, som både kan køre autonomt og med en Playstation Dualshock 4. Projekts krav er netop udmundet i robotbilens krav, som er tre konkurrencer: Sumo, wall-follow og fodbold. Sumo og wall-follow er med hen holdvis IR-sensor og TOF-sensor. Den sidste konkurrence er en fodbold-konkurrence, som styres med den forrige nævnte Playstation Dualshock 4.

Selvom, at projektet fremstilles igennem konkurrancer, så er projekts formål at udvikle vores viden om netværksprotokoller, indlejret systemer, programmering og projektstyring. Samt, hvordan disse integreres til et samlet projekt.

Robotbilen er således også under restriktive, så som at bruge et givet chassis, 1 TOF, 1 IR, 2 motor, 1 Raspberry pi pico w1 og 1 H-bro.

Derved er problemformulering for robotbil-projektet at bygge en robotbil, som kan udføre de tre konkurrencer

6. Metoder og værktøjer

Vi har planlagt projektet efter en traditionel projektmodel med WBS, Gantt-diagram og analyse af den kritiske vej (critical path). WBS opdeler projektet i mindre opgaver, mens Gantt-diagrammet viser opgavernes varighed og rækkefølge. Den kritiske vej er den længste kæde af afhængige opgaver og bestemmer projektets samlede varighed. Metoderne giver overblik og viser, hvilke opgaver der kan forsinke projektets afslutning.

Vi har brugt ClickUp til at organisere opgaver, følge deres status og koordinere arbejdet i gruppen. Opgaver og fremdrift skal opdateres løbende, så overblikket afspejler det faktiske arbejde.

Vi har udviklet softwaren i Python med PyCharm som udviklingsmiljø. Python gør det muligt at opdele koden i funktioner og moduler, så den er lettere at overskue og teste. Vi har brugt flowcharts til at vise programforløb og beslutninger. De dokumenterer logikken, men skal suppleres med test for at kontrollere, at koden fungerer.

Projektet er blevet forsinket og følger derfor ikke længere den oprindelige tidsplan. Gantt-diagrammet og analysen af den kritiske vej skal opdateres ud fra de resterende opgaver og deres afhængigheder. Forsinkelserne viser, at vi skal følge tættere op på fremdriften og vurdere opgavernes tidsforbrug mere realistisk.



7. Projektmiljøet

Opgavens projektmiljø er baseret på en gruppekontrakt udformet før projektets opstart. Gruppekontraktens formål er at skabe et fælles mål, forventningsafstemning og skabe rammer for hvordan projekt skal udføres.

Disse rammer kommer i form af at aftale hvor mange timer, som gruppen bruger på projekt. Samt hvornår og hvem laver hvad. Disse rammer sikker et godt fundament for klar kommunikation og et vellykket projekt.

Samtidig med rammerne danner et fælles fundament, så er forventningsafstemning for hvad hvert gruppemedlem ønsker at opnå også vigtigt. At have et mål, giver projekt en retning og med en retning gør det at, på trods af vanskeligheder og uenigheder i løbet af projekt. Med vores gruppekontrakt og en forventningsafstemning har været med til at mitigerer konflikter i løbet af projekter

Samtidig, som nævnt i kapitel (eller afsnit?) 6, har vi brugt ClickUp til at planlægge projekts forløb og udarbejde diagrammer over projektet. Clickup, skaber et fællesrum og er med til at danne rammer for projektet.



8.Krav

Projektledelse

Projektet skal planlægges ved hjælp af en Work Breakdown Structure (WBS), der opdeler arbejdet i overskuelige opgaver, samt et Gantt-diagram, der viser projektets tidsplan.

Software

Softwaren skal baseres på en modulstruktur med et tydeligt funktionshierarki og udvikles med fokus på stabilitet og testbarhed. Den skal dokumenteres gennem flowcharts, beskrivelser af input, proces og output samt test af funktionaliteten. Hver studerende skal udvikle mindst én softwarefunktion, og holdet skal udarbejde en samlet tegning af hele modulstrukturen med det tilhørende funktionshierarki.

Embedded

I embedded-delen skal der arbejdes struktureret på modulniveau og udarbejdes både et blokdiagram over hele systemet og et elektrisk diagram over bilen. Systemet skal omfatte motorstyring med en DC-motor og en H-bro samt anvendelse af reflektionssensor og TOF-sensor. Energiforsyningen skal ske gennem et LiPo-batteri, og bilen skal kunne fjernstyres via UDP over Wi-Fi. Derudover skal de elektriske og mekaniske problemer, der opstår under projektet, håndteres og løses.

Konkurrence

Derudover skal robotbilen kunne deltage i tre konkurrencer: Wall Follow, SUMO Battle og fjernstyret fodbold. Konkurrencerne stiller forskellige krav til bilens autonome funktioner, sensorer og fjernstyring. I tvivlstilfælde er underviserens afgørelse endelig.

Wall follow

I Wall Follow skal bilen autonomt køre fremad og følge væggen på sin højre side set i bevægelsesretningen. Dette skal ske ved hjælp af TOF-lasersensoren. Konkurrencen gennemføres som tidskørsel på en bane, og der må ikke gribes ind, efter bilen er startet. Der gives point efter, hvor hurtigt banen gennemføres, hvilket kræver, at bilen reagerer tilstrækkeligt hurtigt på ændringer i væggens forløb. Væggen kan have både indadgående og udadgående hjørner på maksimalt 90 grader

Sumo

I SUMO Battle skal bilen autonomt skubbe papkasser af varierende størrelse ud af en cirkulær arena, som er afgrænset med sort tape. Bilen skal anvende sine indbyggede sensorer til navigation, og der må ikke gribes ind efter start. Hvis bilen kører ud af arenaen, må den dog sættes tilbage i midten. Der må ikke tilføjes dele, som rækker ud over bilens chassis. Pointene afhænger af antallet af kasser, der skubbes ud, og den anvendte tid. Der registreres dog kun en tid, hvis alle kasser er skubbet ud af arenaen. Hoveddommeren kan stoppe forsøget, hvis det vurderes, at det ikke er muligt at gennemføre.



Fodbold

Til fjernstyret fodbold skal der udvikles et Python-program på en computer, som giver brugeren mulighed for at styre robotbilen under kampen. Fodboldbanen bliver cirka 4×6 meter med sidevægge og et mål på cirka én meters bredde i hver ende. Kampene afvikles i runder på cirka fem minutter, og hver robotbil skal deltage i mindst tre runder. Python-programmet skal desuden kunne sætte bilen i både Wall Follow-tilstand og SUMO-tilstand samt vise batteriets aktuelle spænding.

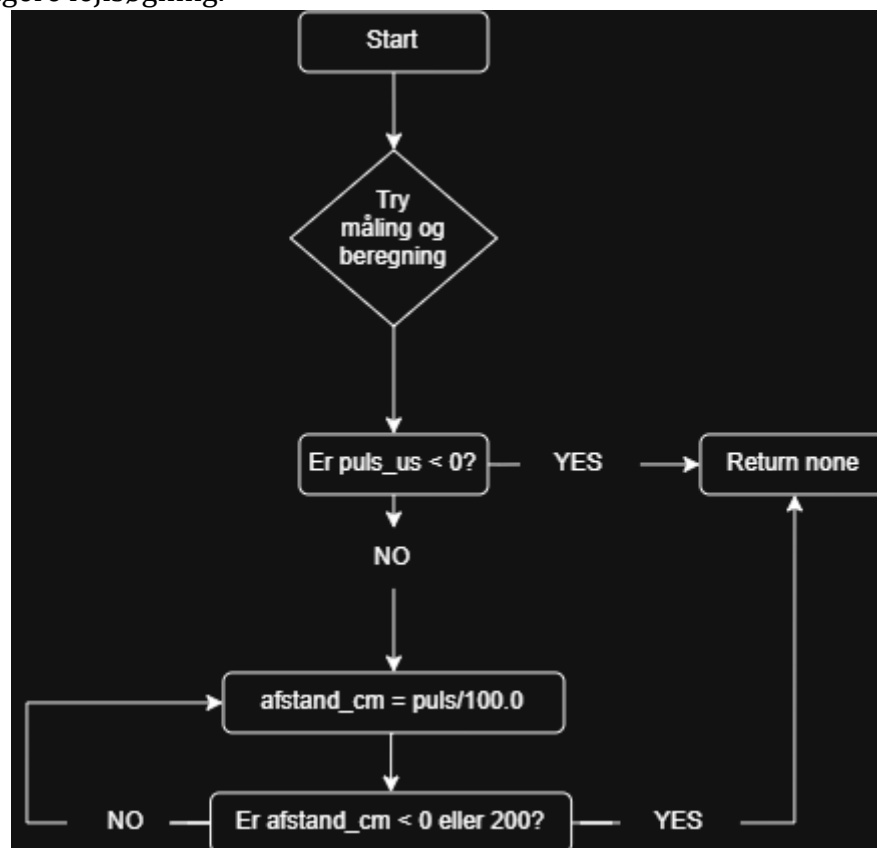


9. Systemdesign og Moduler

9.1.1 TOF-design

ToF-sensormodulet er en central del af projektet, da bilens styring anvender afstandsmålingerne som grundlag for sine beslutninger. Det er derfor vigtigt at reducere påvirkningen fra fejlagtige og afvigende måleværdier. Modulet er designet til at indsamle fem afstandsmålinger og beregne medianen, inden resultatet sendes videre til Pico'en. Denne metode kaldes medianfiltrering og begrænser påvirkningen fra enkeltstående afvigende målinger, så styringen får et mere pålideligt beslutningsgrundlag.

Medianfiltreringen kan dog ikke fjerne alle målefejl. Hvis størstedelen af målingerne i en måleserie er fejlbehæftede, kan medianværdien også blive misvisende. Vedvarende målefejl kan indikere problemer med f.eks. sensoren, dens placering eller måleforholdene og kræver derfor yderligere fejlsøgning.



9.1.2 Implementering

Vi har implementeret dataindsamlingen fra ToF-sensormodulet ved hjælp af en for-løkke, der gennemløbes fem gange. Ved hver iteration foretages én afstandsmåling, hvorefter måleværdien tilføjes en liste med metoden `append()`. Når løkken er afsluttet, beregnes medianen af de fem måleværdier. Herefter tømmes listen, så den er klar til næste måleserie, og medianværdien



returneres til videre behandling i systemet. Denne fremgangsmåde reducerer påvirkningen fra enkeltstående afvigende målinger.

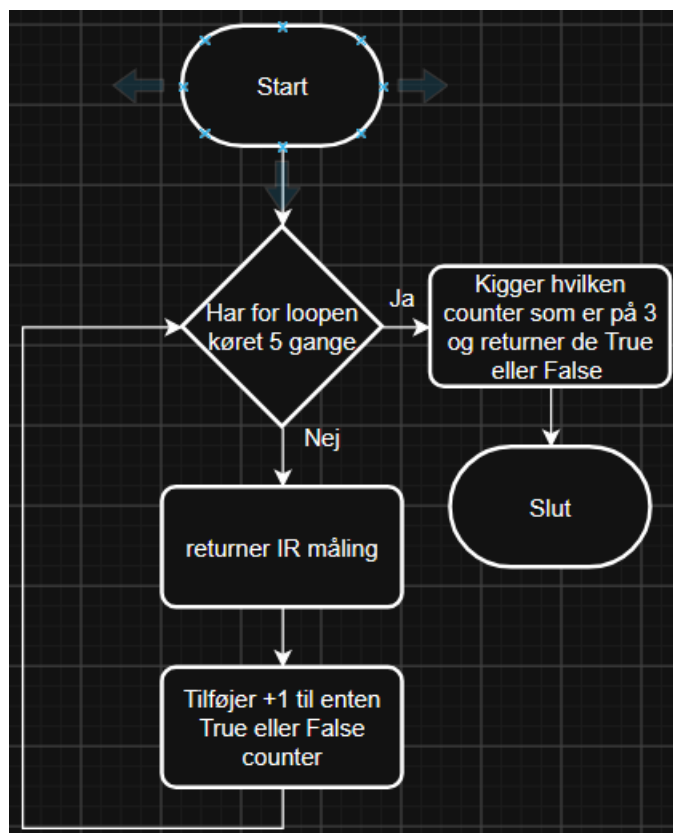
```
def TOF_Task():
    global TOF_List
    for tal in range(TOF_Count):
        distanceTOF = TOF_GetDistance()
        print(f"Dist: {distanceTOF} cm")
        TOF_List.append(distanceTOF)
    med = statistics.median(TOF_List)
    TOF_List.clear()
    return med
```

9.1.3 Modultest

Under test-fasen af selve modulet virkede den overstående kode uden problem og returnerer præcis hvad vi forventede. Der opstod dog et problem under kørsel af wall-follow. Her oplevede vi at programmet målte utroligt langsomt og skabte forsinkelser i hele koden. F.eks. når vi skulle skifte modes var der en forsinkelse på 5-10 sekunder før den skiftede. Derfor valgte vi at ændre koden så vi kunne tage målinger hurtigere, i stedet for at tage 5 målinger og finde medianen kigger vi på hvor langt tid TOF-sensoren er aktiv og omregner det til centimeter. Derefter kigger den på tallet og om det er indenfor grænserne ellers returner den none. De ændrede resulterede vi fik, gjorde at alle vores programmer kørte meget mere glidende og fjernede de forsinkelser vi havde.

9.2.1 IR-design

Ligesom TOF-sensoren er IR-sensoren utrolig vigtig for bilen, den hjælper med at fortælle bilen om den er kørt ud af til kanten af sumo-banen. Det er også vigtigt at vi sikrer en ordentlig måling så den ikke tror vi er ved kanten af sumo-banen selv om den ikke er. Her bruger vi samme design princip som TOF-sensoren hvor der bliver taget flere målinger før den giver et aktuelt svar om den er eller ej.



9.2.2 Implementering

Modulet til IR-sensoren ligner meget TOF-sensoren hvor vi bruger en for loop til at tage 5 målinger som tilføjer en +1 til enten Truecounter eller Falsecounter der tæller op hvilken den skal returne siden IR sensoren kun kan være 1 eller 0.



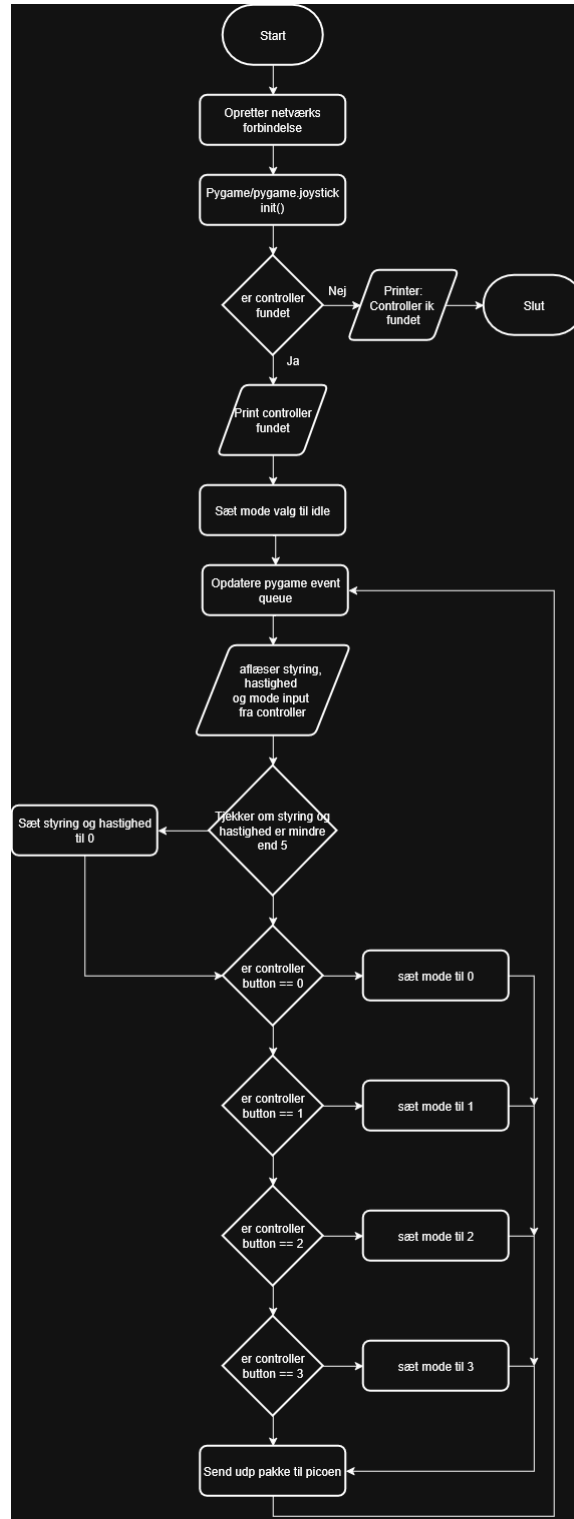
```
def IR_Task():
    global trueCounter, falseCounter
    for tal in range(loopCount):
        Status = IR_GetValue()
        if Status == True:
            trueCounter += 1
        else:
            falseCounter += 1
        if trueCounter == 3:
            trueCounter = 0
            falseCounter = 0
            return True
        elif falseCounter == 3:
            trueCounter = 0
            falseCounter = 0
            return False
```

9.2.3 Test

Der blev kørt en lille test som skulle se hvad den vil return baseret på hvad den kan se. Her forventes der når man peger det på sort tap returneres false, hvis ser noget returneres true. Med lidt justeringer på IR-sensoren fik vi de forventet resultater.

9.3 UDP og Fodbold

9.3.1 UDP og Fodbold Design





Fodboldprogrammet laves lidt anderledes i forhold til sumo og wall follow som skal kunne køre autonomt, her skal vi nu kunne styre robotbilen selv med en controller. Picoen kan ikke tage direkte input fra en controller både wired og wireless derfor har vi skulle sende input over netværket oversat til noget picoen kan forstå imens vi skal prøve at minimere input delay så meget som fysisk muligt. Derfor bruger vi pygame, udp pakker over netværket og en ekstra pc som sender det oversatte input til picoen.

9.3.2 Implementering

Modulet `pc_afsender.py` bruger vi på computeren til at sende kommandoer fra vores controller til Pico W. Her bruger vi `pygame` til at læse controlleren og `socket` til at sende data over netværket med UDP. Programmet tjekker først, om der er en controller tilsluttet, og stopper, hvis det ikke kan finde en. Derefter bruger vi en `while True`-løkke til hele tiden at læse joystickets position, som bliver lavet om til værdier mellem -100 og 100 for styring og fart. Vi har også en deadzone, hvor små udsving under 5 bliver sat til 0, så joysticket ikke sender små bevægelser, når vi slipper det. Knapperne bruger vi til at vælge mode, hvor kryds vælger 0 til IDLE, cirkel vælger 1 til fodbold, firkant vælger 2 til SUMO og trekant vælger 3 til Wall Follow. De tre værdier bliver samlet i beskeden `mode, styring, fart` og sendt til Pico W. Til sidst venter programmet 0,05 sekunder, før løkken starter forfra.

9.3.3 Test

Vi har test og implementeret UDP for True-wireless til at forbinde vores Playstation 4 Dualshock controller til vores Raspberry Pi pico w 1. Samtidig bruges UDP også til at styre, og kontrollerer, samt sende data til vores Raspberry Pi Pico W 1.

9.4.1 Motor

9.4.2 Implementering

Modulet `motor.py` bruger vi til at styre bilens to motorer, hvor vi har klasserne `Motor` og `RobotDrive`. Klassen `Motor` styrer én motor, hvor vi bruger to pins til at bestemme retningen og en PWM-pin til at justere hastigheden. Funktionen `set_speed()` begrænser hastighedsværdien til mellem -100 og 100, hvor positive værdier er fremad, negative værdier er baglæns, og 0 stopper motoren. Vi bruger `abs()` til at finde værdien uden fortegn og omregner den til en PWM-værdi mellem 0 og 65535, som bestemmer, hvor stor en del af tiden signalet er tændt. I klassen `RobotDrive` opretter vi en venstre og en højre motor, som vi styrer gennem funktionen `drive()`. Her ganger vi hastighederne med hver sin trimværdi, som er 0,95 for venstre motor og 0,60 for højre motor, så vi kan kompensere for forskelle mellem motorerne. Vi har også funktionerne `forward()`, `backward()`, `turn_left()` og `turn_right()`, så vi nemt kan vælge bilens bevægelse. Når bilen skal dreje, kører hjulene i hver sin retning, så den kan dreje på stedet. Til sidst har vi funktionen `stop()`, som stopper begge motorer, og hvor vi kan vælge, om der skal bruges aktiv bremsning.

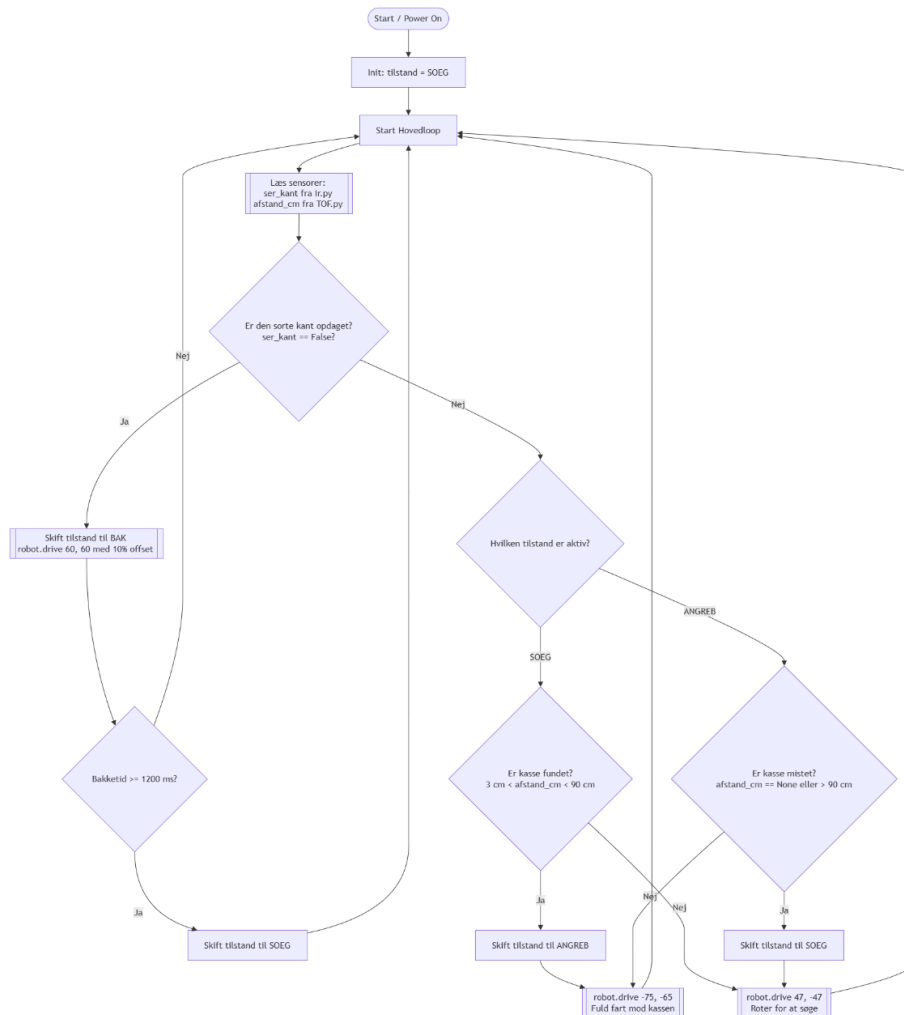
9.4.3 Test

Motoren modulet er test, og op at køre. Motor-modulet bruges, som fundament for alle efterfølgende moduler, sumo, fodbold og wall-follow. Derved, kan det igennem testen



observeres at den virker, og det kommer til udtryk i for eksempel til udtryk i fodbiltest video i afsnit 9.7.3.

9.5.1 Sumo design



9.5.2 Implementering

Modulet Sumo er bygget op som en tilstandsmaskine. Sumo har fem tilstande. "SOEG", "SWEEP", "DREJ MOD MIDTE", "BAK" og "ANGREB". Sumo programet får input fra TOF-sensor og IR-sensor. Programmet bruger ikke-blokerende tidsstyring i form af `time.ticks_ms`. Sumo programmet starter i tilstand "SOEG" og derfra kører programmet IF/ELIF tilstande. 1 prioritet er at reagere på IR-sensor registrerer den sorte tape der afgrænser sumo arenaen. Det vil sige at IR-sensor kant-detektion afbryder enhver tilstand på nær "BAK". Hvis der er kant-detektion, så bakker robotbilen i 1200 ms og overgår derefter til tilstand "SOEG" igen. Hvis TOF-sensor får et input imellem 2 cm og 90 cm så overgår programmet til tilstand "SWEEP" og gemmer klokken. Idet TOF-sensor får input over 120 cm regnes papkassen for at være ude af synsfelt, programmet gemmer igen klokken, finder differencen på de to tal, dividerer med 2 og skifter til tilstand "DREJ MOD MIDTE". Robotbilen drejer differencen tilbage så den gør front mod midten og overgår til tilstand "ANGREB". I tilstand "ANGREB"



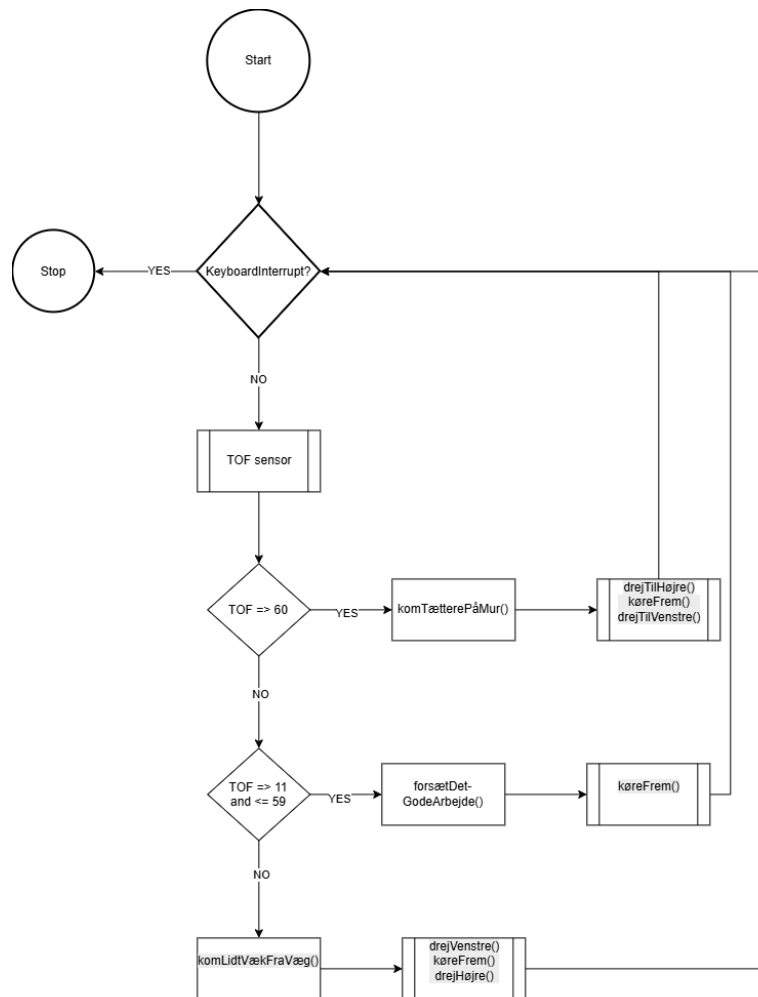
kører bilen direkte mod midten af kassen, indtil den registrerer den sorte tape. Hvis den mister papkassen under "ANGREB", overgår robotbilen til tilstand "SOEG" igen.

9.5.3 Test

I testfasen havde robotbilen svært ved at finde papkasserne og derved midten. Det resulterede i at robotbilen kørte ind imellem papkasserne. For at finde ud af hvor i koden problemet var ændre vi hastigheden i tilstand "SWEEP" så når den overgik til denne tilstand så ville vi se at robotbilens hastighed ville stige og hvis hastigheden ikke steg, så vidste vi at den ikke kom i tilstand "SWEEP". Det blev hurtigt tydeligt for os at robotbilen ikke registrerede venstre kant af papkasserne fordi den først øgede hastigheden idet at TOF-sensor var ud for højre kant af papkassen. Vi forsøgte derefter at ændre TOF modul så ledes at den skulle tage median af færre målinger for at få programmet til at køre hurtigere. Vi forsøgte at forbinde TOF-sensor og PICO via I2C i stedet for PWM. Det lykkedes men TOF-sensor blev ikke bedre til at registrerer papkasserne. Vi forsøgte at få bilen til at søge mod uret i stedet for med uret og vi forsøgte at sænke hastigheden så meget som vi kunne, men ingenting hjalp. Til sidst fjernede vi tilstand "SWEEP" så ledes at robotbilen skulle gå direkte i tilstand "ANGREB". Det resulterede i at robotbilen angreb kasserne og at programmet næsten kørte fejlfrit. Dog kunne robotbilen blive fanget i et loop hvor den kørte frem og tilbage. Det løste vi ved at sænke hastigheden med 10% på venstre motor så bilen fik drejet sig en lille smule efter hver angreb i forbindelse med når den bakkede.



9.6.1 Wall follow



9.6.2 Implementering

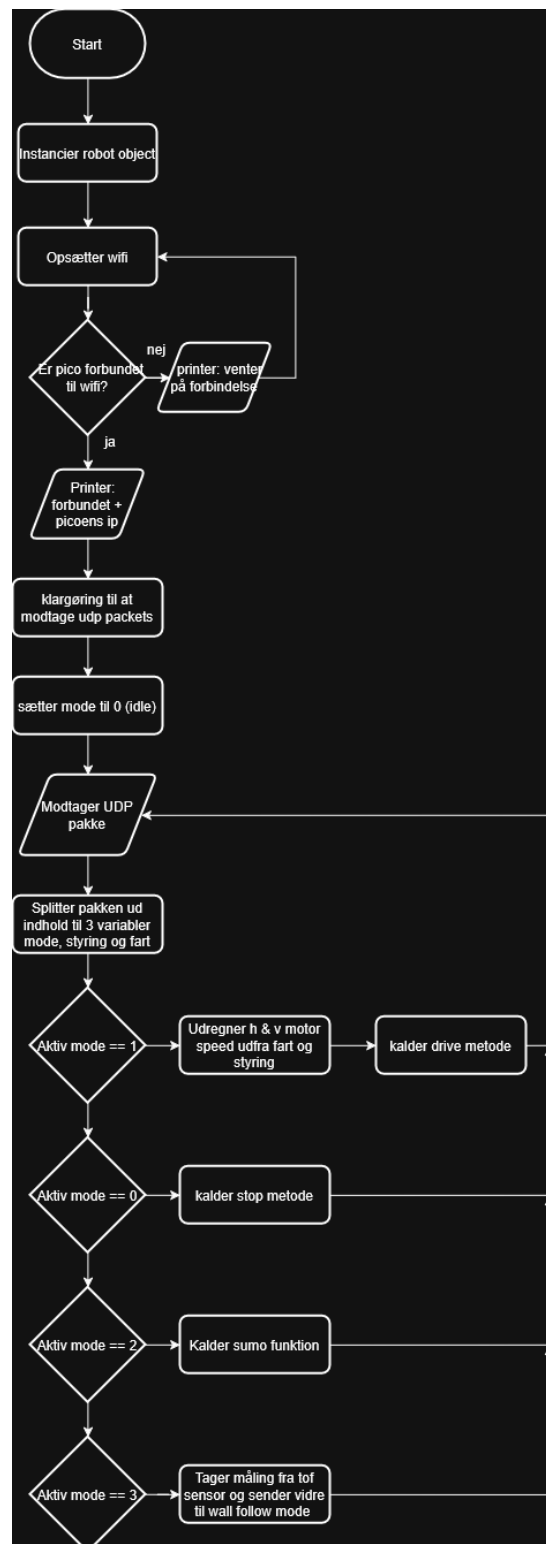
Modulet til Wall Follow bruger afstanden fra TOF-sensoren på højre side af bilen til at holde en afstand på 20 cm fra væggen. Her bruger vi funktionen `wall_follow_step()`, hvor vi sender robotobjektet og den målte afstand ind. Vi finder fejlen ved at trække de 20 cm fra målingen og ganger derefter fejlen med 2,5 for at finde, hvor meget vi skal justere motorerne. Hvis bilen er for langt fra væggen, kører venstre hjul hurtigere end højre, så bilen drejer ind mod væggen. Hvis bilen er for tæt på, gør vi det modsatte. Vi bruger også funktionen `constrain()` til at holde hastighedsværdierne mellem 15 og 70. Hvis målingen er ugyldig eller over 50 cm, får venstre hjul værdien 35 og højre hjul værdien 10, så bilen drejer mod højre og søger efter væggen. Funktionen skal kaldes igen med en ny måling, hver gang vi vil justere bilens kørsel.

9.6.3 Test

Dette er [Wall follow](#), som dokumenterer at robotbilen kan køre autonom og uden input fra en kontroller. Testen viser, at bilen at robotbilen kan køre fra a til b.



9.7.1 Main design





Main er rykraden på vores robotbil, det er den som skal kunne gøre vi kan skifte til de forskellige modes uden at man skal stoppe bilen loade nyt programmer på i stedet. Siden vi ikke kan have et kabel tilkoblet under robotkonkurrencen, er det vigtig at kunne skifte hvilken mode der skal være aktiv trådløst. Tidligere i UDP-modulet blev der nævnt vi afsender oversatte instrukser til picoen over netværket, her bruger vi main til at skilde de her UDP-pakker fra hinanden så de bliver til 3 variabler aktiv mode, styring og fart som vi kan bruge til at kontroller robotbilen med fra en kontroller.

9.8.2 Implementering

```
# 1. Init hardware og sensor-interrupts
robot = RobotDrive(left_trim=1.0, right_trim=1.0)
TOF.TOF_GetDistance_Fast()

# 2. Wi-Fi Opsætning
wlan = network.WLAN(network.STA_IF)
wlan.active(True)
wlan.connect(ssid: "ITEK 2nd", key: "2nd_Semester_E25")

while not wlan.isconnected():
    print("Venter på Wi-Fi forbindelse...")
    sleep_ms(500)

print(f"Pico W er på netværket! IP: {wlan.ifconfig()[0]}")

# 3. UDP Socket
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.bind(("0.0.0.0", 5005))
sock.setblocking(False)

print("Klar til at modtage UDP-pakker...")
```



```
# 4. Hoved-løkke
aktiv_mode = 0

while True:
    try:
        data, addr = sock.recvfrom(1024)
        tekst = data.decode("utf-8")

        if "," in tekst:
            del_mode, del_styring, del_fart = tekst.split(",")
            aktiv_mode = int(del_mode)
            styring_val = int(del_styring)
            fart_val = int(del_fart)

            # Mode 1: PS4 / Manuel styring
            if aktiv_mode == 1:
                v_speed = max(-100, min(100, fart_val + styring_val))
                h_speed = max(-100, min(100, fart_val - styring_val))
                robot.drive(v_speed, h_speed)

        except (OSError, ValueError, IndexError):
            pass

        # --- Mode håndtering ---
        if aktiv_mode == 0:
            robot.stop()

    elif aktiv_mode == 2:
        # Autonom Sumo
        #afstand_cm = TOF.TOF_GetDistance_Fast()
        sumo.kor_sumo_step(robot)
        #print(f"Dist: {afstand_cm} cm")
        # Tjek hvad den siger midt på den hvide plade:
        #print("Hvid plade test (skal være False):", ir.IR_Task())

    elif aktiv_mode == 3:
        try:
            afstand_cm = TOF.TOF_GetDistance_Fast()

            wall_follow.wall_follow_step(robot, afstand_cm)

        except Exception as e:
            pass

    sleep_ms(10)
```



Koden til main består af 2 dele, det første som sker, er at robotobjektet bliver oprettet som bruges til at kontrollere motorerne så kan køre delvist synkront. Derefter bliver alt netværks mæssigt oprettet samt udp modtagning klargøres. Den anden del af main er while loopet som køre. Dens opgave er at kigge på udp pakken som pico'en modtager og splitte den op i tre variabler ved at bruge split til at fjerne alle kommaer og sæt de 3 strings lige med hver sin variable. Efter dette bliver de strings lavet som til integers så de kan bruges til at regne med. Her til sidst kigger den på hvilket mode er aktiv ved at se hvilket tal aktiv mode er, hvor den så kalder tilhørende funktion under den aktiv mode.

9.8.3 Test

Main har kørt upåklageligt under hele testfasen. Det eneste man skal være opmærksom på er at have kabel i PICO hver morgen når man starter op og kontrollere den IP-adresse PICO bliver tildelt. For det skal være den samme som der er i afsender.py ellers kan PC ikke sende UDP signaler til PICO.



10. Systemtest/integrationstest

I forbindelse med integrationstest har vi haft lidt navne forvirring ved import sætninger, variabel navne samt kald af den helt korrekt stavede funktioner og moduler. Det var i småtingsafdelingen men når man har 6-7 python filer der snakker sammen så kan det hurtigt blive lidt komplekst.

I forbindelse med systemtest blev det observeret at vores robotbil ikke kørte lige ud under "ANGREB" i Sumo mode. Vi valgte bevidst ikke at indføre den globale trim-justering, da vi var nervøse for at det ville påvirke de andre modes i vores program. I stedet for justerede vi hvor meget kraft hver fik inde i Sumo koden. I forlængelse af dette var vi også meget påpasselig med at rette i vores TOF-modul og IR-modul. Sumo var det sidste program vi testede og vi vidste at vi var nødt til at starte vores test forfra i den andre programmer hvis vi ændrede i vores moduler.



11. Bevis

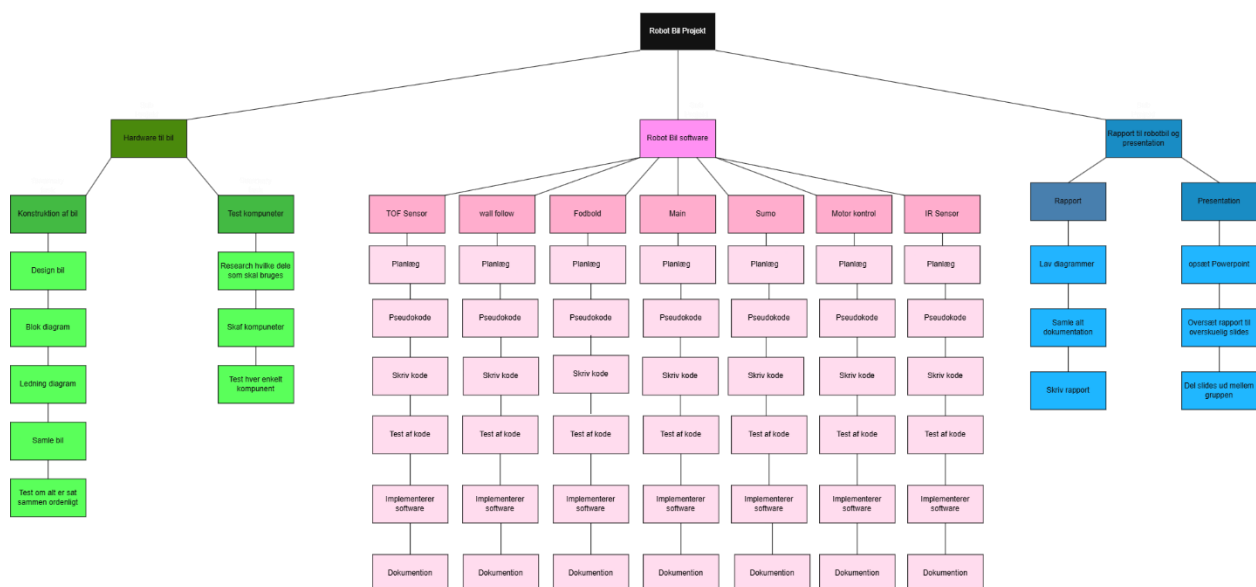
[Wall-follow](#)

[Sumo](#)

12. Projektledelse

Som en del af robotbilen, er der blevet udarbejdet et Work Breakdown Structure (WBS) diagram. WBS er metodisk og hierarkisk arbejdsform. Hvis formel er at nedbryde projektet i mindre og overskuelige dele. WBS er et fundamentalt kort til at forstå hvad der skal bygges.

Med WBS kan projektet brydes ned til sub-projekter, og i robotbilen er der tre sub-projekter. Hardware til bilen, software og rapport. Under et sub-projekt er der summary tasks. Hver summary task er noget, som skal laves indenfor 4-5 timer. I robotbils projektet er software sub-task, den største og mest gentagende.



Derudover er der også brugt et Gantt-diagram, (bilag 1) til at uddybe og tidsberette WBS-diagrammet. Formålet med et Gantt-diagram er skabe overblik over projektet, samt også at give projekt en retning fra a til b. Netop at Gantt-diagrammet er vigtig for robotbilens succes i og med at det skaber en konkret retning, tidslinje og klare opgaver, som også er optegnet i WBS-diagrammet.



13. Perspektiv / refleksioner

Projekts proces bestod, af en hurtig opdeling af studiegruppen. Hvor den ene, fokuseret at få bilen bygget, den anden startet med at programmerer koden. Netop dette, og uden en fælles forståelse eller klar retning for hvad og hvordan bilen skulle udformes og se ud.

I løbet af processen, brugt vi for meget tid på at bygge bilen. Denne spildtid betød at vi kom bagud. Dette er en vigtig erfaring for fremtidig projekter, at vi skal stramme op på projekts udseende og konstruktion. Fremfor at hoppe direkte ind og starte bygge, så ville det være mere optimalt at sætte bremsen i og starte med at brainstorme ideer. Samt at skitse det generelle design før man bygger selve projektet.

I løbet af dette projektforsløb har vi erfaret at, for at få et streamlined projekt, så skal vi være bedre til at lave forventingsafstemning. Samt også, at vi som studiegruppe har en tendens til at gå hjem for tidlig og ud skubbe projekt til en senere dato. Det er også nået, som til fremtidens projekter at vi skal stramme op på.



14. Konklusion

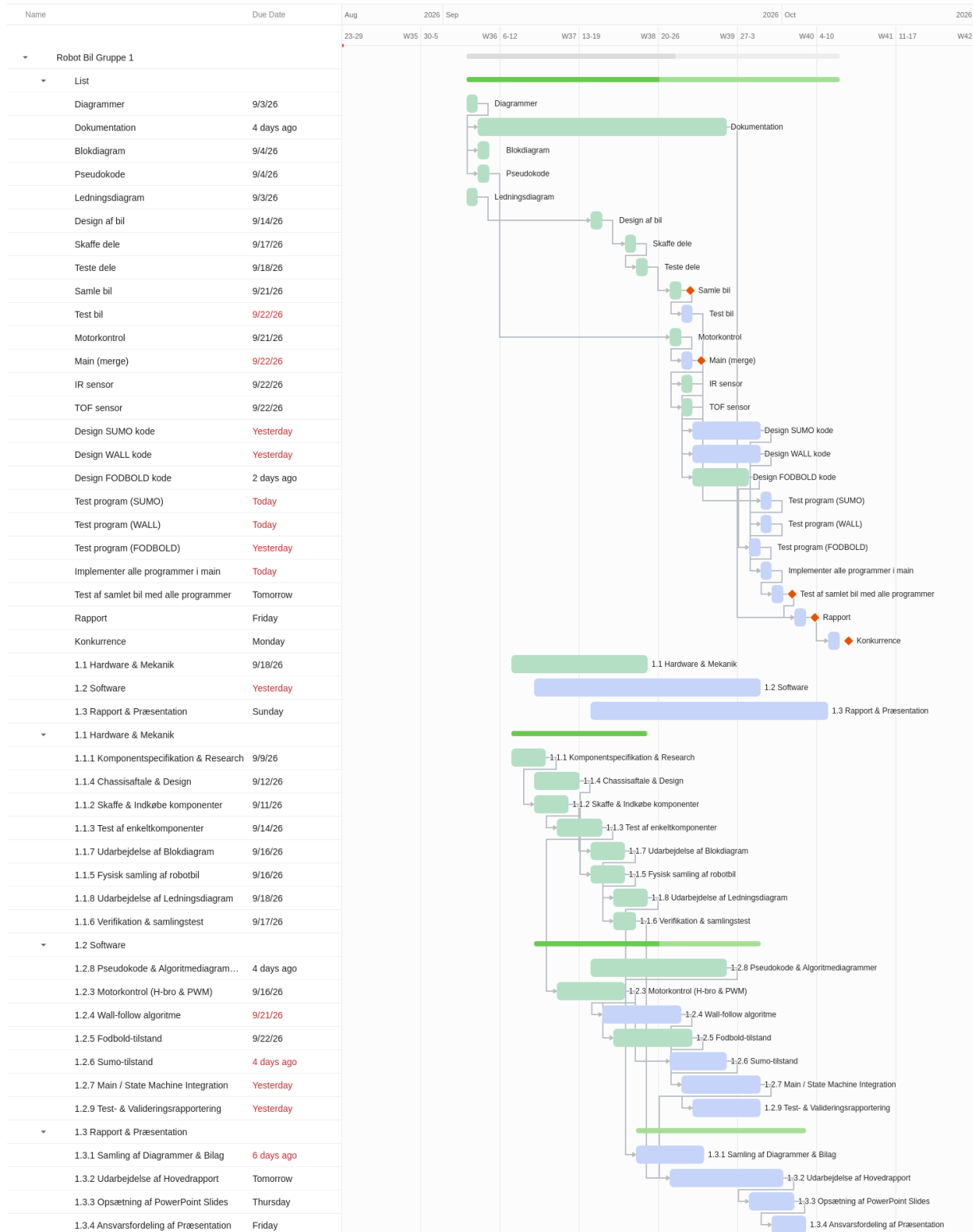
I projektet har vi arbejdet med at udvikle en robotbil til konkurrencerne Wall Follow, SUMO og fjernstyret fodbold. De vigtigste krav var, at bilen skulle kunne køre autonomt ved hjælp af TOF- og IR-sensorer samt fjernstyres gennem et Python-program via UDP over Wi-Fi. Programmet skulle desuden kunne skifte mellem kørselsfunktionerne og vise batteriets aktuelle spænding. Løsningen skulle bygges med de udleverede komponenter og have en modulopbygget softwarestruktur.

Vores løsning er bygget op omkring en Raspberry Pi Pico W, som bruger sensormålinger og kommandoer fra computeren til at styre motorerne gennem en H-bro. Softwaren er opdelt i moduler til blandt andet sensorer, motorstyring, kommunikation og de forskellige kørselsfunktioner. Det har gjort det muligt at udvikle og teste funktionerne hver for sig. De beskrevne tests viser, at sensorerne og motorstyringen fungerer, at bilen kan fjernstyres med en PlayStation-controller gennem computeren, og at den kan køre autonomt med Wall Follow. Vi har dermed udviklet en løsning på centrale dele af problemformuleringen, mens opfyldelsen af alle konkurrencekrav endnu ikke er dokumenteret.

Projektprocessen har omfattet planlægning, konstruktion, programmering og test. Vi har anvendt WBS, Gantt-diagram og ClickUp til at opdele opgaverne og skabe overblik. Konstruktionen tog længere tid end forventet, hvilket medførte forsinkelser. Forløbet har derfor vist betydningen af et fælles design, en tydelig opgavefordeling og løbende opfølgning på tidsplanen. Arbejdet har samtidig givet os praktisk erfaring med, hvordan programmering, netværkskommunikation, elektronik og mekanik skal fungere sammen i et samlet system.

15. Appendiks

Robot Bil Gantt Diagram





Gantt diagram 1